
Fine-Tune Vision Transformers for Image Classification on a Small Dataset

Jiahe Guo

Autonomous Systems and Intelligent Robotics
KTH Royal Institute of Technology
slept24hours@gmail.com

Abstract

This thesis explores the effectiveness of fine-tuning Vision Transformers on small-scale image classification tasks, using the Oxford-IIIT Pet Dataset as a benchmark. While ViTs have achieved leading performance on large datasets, their performance on limited data remains underexplored. I evaluated several fine-tuning strategies, including freezing base layers, gradual unfreezing, differential learning rates, and data augmentation. Experimental results showed that despite these efforts, fine-tuned ViTs perform no better than convolutional neural networks such as ResNet18 and ResNet50 on the same task. This suggests that ViTs may not be well-suited for small datasets without significant architectural or regularization adjustments. The findings emphasize the importance of model selection based on dataset size and complexity in transfer learning scenarios.

1 Introduction

The advent of Vision Transformers (ViT) has revolutionized computer vision by adapting the transformer architecture's success in natural language processing to image recognition tasks. However, a critical gap exists in understanding their performance characteristics on very small datasets, which represent a common real-world scenario where data collection is expensive or constrained.

This research addressed the fundamental question: Can ViTs be effectively fine-tuned to achieve competitive performance on small-scale image classification tasks? I investigated this through comprehensive experimentation on the Oxford-IIIT Pet Dataset, comparing various fine-tuning strategies against established convolutional baselines.

The motivation for this study stems from the practical need to understand when ViTs provide value over traditional approaches, particularly given their computational requirements and training complexity. My findings informed transfer learning strategies and provide guidance for practitioners working with limited datasets.

2 Related Work

How to Train Your ViT? Data, Augmentation, and Regularization in Vision Transformers, Steiner et al. [1] combining increased computational resources with augmentation and regularization (AugReg) enables models to achieve performance comparable to those trained on significantly larger datasets. Their work showed that ViT models trained on ImageNet-21k could match or exceed the performance of counterparts trained on the substantially larger JFT-300M dataset. However, limited research had specifically addressed the challenges of fine-tuning ViTs on truly small datasets.

3 Data

3.1 Dataset

I utilized the Oxford-IIIT Pet Dataset, which is a 37-category pet dataset with roughly 200 images for each class, totaling 7349 images. The images have a large variations in scale, pose and lighting. It is an ideal small-scale dataset for my investigation.

Following the official split, I used 3,680 images for training and 3,669 for testing. To enable proper validation during training, I further split the training set into 80% for training and 20% for validation. All images were resized to 224×224 pixels to match the input requirements of my pre-trained models.

3.2 Model

The ViT model was pre-trained on ImageNet-21k (14 million images, 21,843 classes) at resolution 224×224 , and fine-tuned on ImageNet 2012 (1 million images, 1,000 classes) at resolution 224×224 . It was introduced in the paper *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* by Dosovitskiy et al. [2] and first released in this repository [3]. However, the weights were converted from the `timm` repository by Ross Wightman, who already converted the weights from JAX to PyTorch. Credits go to him.

The model architecture consists of 12 transformer encoder layers, each containing multi-head self-attention mechanisms and feed-forward networks. For my classification task, I replaced the original classification head with a new linear layer outputting 37 classes.

4 Methods

4.1 Fine-tuning Strategy

My approach implemented a progressive fine-tuning methodology designed to mitigate catastrophic forgetting while enabling effective adaptation to the target domain.

I began by freezing all transformer encoder layers and training only the newly initialized classification head. This approach leveraged the pre-trained feature representations while preventing overfitting on the small dataset.

Then starting from epoch 5, I progressively unfroze one additional encoder layer every 2 epochs. This gradual approach allows the model to adapt hierarchically, with earlier layers maintaining general feature extraction capabilities while later layers specialize for the target task.

I implemented a two-tier learning rate system where the backbone ViT layers use a learning rate 10 times smaller than the classification head. This ensured conservative updates to pre-trained features while allowing rapid adaptation of the task-specific components.

4.2 Optimization and Regularization

I employed the AdamW optimizer, which has proven effective for transformer training.

I implemented a cosine annealing schedule with warm-up, where the learning rate gradually increases during the first 10% of training steps before following cosine decay. This approach helps stabilize training and improve convergence.

I applied standard image augmentation techniques including random horizontal flips, random rotation (± 15 degrees), and color jittering to increase data diversity and reduce overfitting.

Weight decay was excluded from the optimization strategy as it demonstrated negligible performance gains and risks of over-regularization when applied to limited-scale datasets [2].

I implemented early stopping based on validation accuracy with a patience of 5 epochs to prevent overfitting.

4.3 Baseline

To provide context for my ViT results, I implemented ResNet baselines, ResNet50 for the full 37-class classification task. This model underwent similar fine-tuning procedures with gradual unfreezing and differential learning rates to ensure fair comparison.

5 Experiments

In my experiment, the training process was deployed on RTX4070 GPUs (8G).

5.1 Learning Rate Optimization

My initial experiments focused on identifying optimal learning rates for the classification head while keeping the backbone frozen. Table 1 presented the results across different learning rate configurations.

Table 1: Classification Head Fine-tuning Results

Learning Rate	Epoch	Train Runtime (s)	Eval Runtime (s)	Eval Accuracy
1e-2	15.0	322.13	18.94	0.9128
1e-3	15.0	308.47	17.32	0.9248
5e-4	15.0	380.26	26.75	0.9278
1e-4	15.0	467.90	26.97	0.9275
5e-5	15.0	461.96	27.06	0.9163

5.2 Gradual Unfreezing

Table 2 presented the effect of implementing gradual unfreezing combined with differential learning rates. This approach allowed deeper adaptation while maintaining stability.

Table 2: Gradual Unfreezing with Differential Learning Rates

Learning Rate	Epoch	Train Runtime (s)	Eval Runtime (s)	Eval Accuracy
1e-2	3.8043	88.55	18.88	0.891
1e-3	7.0652	467.52	18.16	0.9256
5e-4	15.0	545.90	25.75	0.9308
1e-4	15.0	483.24	25.15	0.9338
5e-5	15.0	447.40	25.48	0.927

The gradual unfreezing strategy shows improvement, with the best configuration (1e-4 learning rate) achieving 93.38% accuracy. This represents a 0.6 percentage point improvement over the frozen backbone approach.

5.3 Learning Rate Scheduler

I used the cos learning rate scheduler based on Differential Learning Rates, for the first 10% of the training steps, the learning rate will gradually increase (warm up) from a small value to the initial learning rate before following the cosine decay.

Table 3 showed the impact of implementing cosine annealing with warm-up on my best-performing configurations.

The learning rate scheduler provided marginal improvements in training stability but does not significantly impact final accuracy. The 1e-4 configuration maintains strong performance at 93.35%. The 1e-4 configuration maintains high accuracy with moderate training time.

Table 3: Cosine Annealing with Warm-up Results

Learning Rate	Epoch	Train Runtime (s)	Eval Runtime (s)	Eval Accuracy
1e-2	3.8043	86.93	18.30	0.8828
1e-3	7.0652	155.42	17.63	0.9242
5e-4	14.6739	350.32	17.71	0.9302
1e-4	15.0	363.68	18.51	0.9335
5e-5	15.0	366.88	17.33	0.9272

5.4 Data augmentation

Table 4 presents the results after incorporating data augmentation techniques into my best-performing model configuration.

Table 4: Data Augmentation Integration

Learning Rate	Epoch	Train Runtime (s)	Eval Runtime (s)	Eval Accuracy
1e-2	3.8043	87.56	18.42	0.8883
1e-3	7.0652	155.42	18.01	0.9248
5e-4	14.6739	354.42	17.95	0.9302
1e-4	15.0	364.75	18.40	0.9335
5e-5	15.0	306.52	17.83	0.9272

Data augmentation maintains consistent performance without significant improvement.

5.5 Experiment for ConvNet

Before using convolutional neural networks (ConvNet), I resized the images in the dataset to 224x224.

For the binary classification task of recognizing pictures of dogs vs. cats, I used ResNet18 with a batch size of 64, learning rate of 0.001, 10 epochs, and the Adam optimizer. This achieved an accuracy of 99.2%.

For the 37-class classification task of recognizing dog or cat breeds, I used a larger model, ResNet50, to improve accuracy. I achieved a gradual unfreezing strategy and also applied differential learning rates, a learning rate scheduler, and data augmentation. Specifically, I gradually unfroze all 4 layers after 15, 10, 10, 10, and 10 epochs, respectively, and set learning rates to 0.001, 0.0001, 0.00005, 0.00001, and 0.000001 for the fc, layer4, layer3, layer2, and layer1 layers, respectively. I achieved 94.02% accuracy on the Oxford-IIIT Pet Dataset.

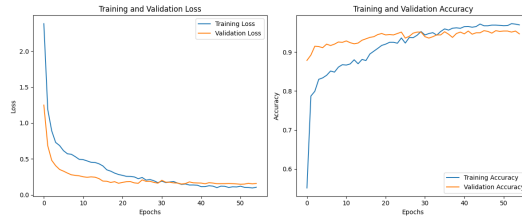


Figure 1: 37-class classification problem of recognising the breed of cat or dog.

6 Conclusion

After optimizing both ViT and ConvNet models, the ViT's performance was even lower than the ConvNet's. The best ViT configuration achieved 93.38% accuracy, while ResNet architectures reached 94.02% on similar tasks.

This study provided empirical evidence that ViTs, despite various optimization strategies, do not achieve superior performance compared to convolutional neural networks on small-scale image classification tasks. My comprehensive evaluation of fine-tuning strategies including gradual unfreezing, differential learning rates, and data augmentation yielded marginal improvements but failed to get better performance than traditional architectures.

These findings suggested that practitioners working with limited datasets should carefully consider the trade-offs between model complexity and data requirements. ViTs are not more effective than convolutional architectures for very small dataset scenarios.

For future research directions, Investigating hybrid architectures combining convolutional and transformer components is a good option.

References

- [1] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your vit? data, augmentation, and regularization in vision transformers, 2022.
- [2] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale - official code. https://github.com/google-research/vision_transformer, 2020.